

eml-sr_fable Feynman 2.test / 2.test_v2 ― 別 PC 実行手順書

対象スクリプト

ファイル	内容
src/feynman_eml_sr_fable_test2.py	緩和条件・クリーンなターゲット・ホールドアウト 500 点
src/feynman_eml_sr_fable_test2_v2.py	test2 と同一条件 + 訓練ターゲットに 1% ガウスノイズ

想定用途: 高性能 PC 上で Feynman 99 式の大規模ベンチマークを実行する。

0. 概要

2.test 系は 1.test より探索資源を拡大した設定である。

パラメータ	1.test	2.test / 2.test_v2
BEAM_WIDTH	1000	2000
N_SAMPLES	750	1500
MAX_COMPLEXITY	6	8
time_budget_s	110	600 (式あたり最大 10 分)
max_boost_terms	6	8

test2 と test2_v2 の違いは **訓練データへのノイズ注入の有無**のみである。

- test2:** クリーンなターゲット。合否はテスト RMSE $< 10^{-4}$ 。
- test2_v2:** 訓練に $y \sim \mathcal{N}(0, (0.01 \cdot \mathrm{std}(y))^2)$ (seed = 777 + 行番号)。合否はクリーンなテスト点に対する RMSE $< \max(10^{-4}, 0.15 \sigma)$ 。

探索ロジック・データ生成シード (42 + 行番号) は 1.test と同一である。

1. 前提ソフトウェア

ソフトウェア	バージョン目安	用途
Git	任意	リポジトリ取得
Python	3.8 以上	ベンチマークスクリプト
Rust (cargo)	最新安定版推奨	eml_sr_fable のビルド
maturin	1.x	Rust → Python 拡張モジュール化

Windows のみ追加: Visual Studio Build Tools の「C++ によるデスクトップ開発」ワークロード (Rust のリンクに必要)。

GPU は不要 (CPU のみ)。

2. リポジトリの取得とブランチ切り替え

test2 / test2_v2 は **main ブランチには未マージ** である。必ず作業ブランチを checkout すること。

```
git clone https://github.com/blabo25226/eml-sr_model.git
cd eml-sr_model

git fetch origin
git checkout claude/eml-sr-fable-algorithm-7w8j49
```

確認:

```
git branch
# * claude/eml-sr-fable-algorithm-7w8j49

ls src/feynman_eml_sr_fable_test2.py
ls src/feynman_eml_sr_fable_test2_v2.py
ls data/FeynmanEquations.csv
ls eml-sr_fable/Cargo.toml
```

clone 時点で揃う主なファイル:

- `data/FeynmanEquations.csv` — Feynman 99 式の定義
- `eml-sr_fable/` — Rust エンジン本体
- `src/feynman_fable_common.py` — 共通ロジック (JSON / Markdown 出力を含む)
- `results/eml_sr_model_*_feynman_results.json` — first_AI / cursor との比較用ベースライン (任意)

3. Python 環境の構築

リポジトリルートで仮想環境を作成し、依存パッケージを入れる。

Linux / macOS

```
python3 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install maturin numpy pandas
```

Windows (PowerShell)

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install --upgrade pip
pip install maturin numpy pandas
```

4. eml_sr_fable のビルド

Rust 拡張モジュール `eml_sr_fable` を release ビルドでインストールする。

```
cd eml-sr_fable
maturin develop --release --features python,full-math
cd ..
```

動作確認:

```
python -c "import eml_sr_fable; print(eml_sr_fable.__file__)"
```

パスが表示されれば成功。 `ModuleNotFoundError` が出る場合は、**activate した venv と同じ環境** で `maturin develop` を再実行する。

(任意) Rust 単体テスト:

```
cd eml-sr_fable
cargo test
cd ..
```

5. スモークテスト (本番前に必須)

src/ ディレクトリから 実行する（`feynman_fable_common` が同ディレクトリにあるため）。

```
cd src

python feynman_eml_sr_fable_test2.py --smoke
python feynman_eml_sr_fable_test2_v2.py --smoke
```

3 式（[I.12.1](#), [I.12.5](#), [I.14.3](#)）のみ実行。数分～十数分で完了する想定。

成功時の出力:

スクリプト	JSON	Markdown レポート
test2	results/eml_sr_fable_feynman_test2_results_smoke.json	texts/eml_sr_fable_feynman_test2_report_smoke.md
test2_v2	results/eml_sr_fable_feynman_test2_v2_results_smoke.json	texts/eml_sr_fable_feynman_test2_v2_report_smoke.md

ターミナル末尾に次のような表示が出れば完了:

```
[INFO] Results saved to ...
[INFO] Report written to ...
```

6. 本番実行 (99 式)

6.1 test2 (クリーン)

```
cd src
python feynman_eml_sr_fable_test2.py
```

6.2 test2_v2 (1% ノイズ)

```
cd src
python feynman_eml_sr_fable_test2_v2.py
```

6.3 先頭 N 式だけ試す

```
python feynman_eml_sr_fable_test2.py 10
python feynman_eml_sr_fable_test2_v2.py 10
```

6.4 実行時間の目安

- 式あたり最大 **600 秒**（早期終了する式も多い）
- 99 式フル実行は **十数時間～1 日以上** の可能性あり
- 長時間実行には `screen` / `tmux`（Linux）やバックグラウンド実行を推奨

Linux の例:

```
cd src
nohup python feynman_eml_sr_fable_test2.py > ../results/feynman_fable_test2.log 2>&1 &
```

Windows PowerShell の例:

```
cd src
python feynman_eml_sr_fable_test2.py 2>&1 | Tee-Object -FilePath ..\results\feynman_fable_test2.log
```

7. 出力ファイル

7.1 スクリプトが自動生成するもの

種類	test2	test2_v2
JSON（機械可読な全結果）	results/eml_sr_fable_feynman_test2_results.json	results/eml_sr_fable_feynman_test2_v2_results.json
Markdown レポート	texts/eml_sr_fable_feynman_test2_report.md	texts/eml_sr_fable_feynman_test2_v2_report.md

Markdown の生成は `test2.py` 内ではなく、`src/feynman_fable_common.py` の `run_benchmark()` → `write_report()` が担当する。

JSON は **1 式終了ごとに上書き保存** される（途中クラッシュしてもそこまでの結果は残る）。

7.2 ターミナル出力の `.log` への保存

`.log` ファイルはスクリプトが自動では書き出さない。実行中にターミナルへ表示される標準出力（式ごとの進捗・RMSE・サマリー）を残すには、次のいずれかを行う。

方法 A: 実行時にリダイレクト（推奨）

6.4 節のとおり、実行と同時にファイルへ書き出す。

方法 B: 計算後にコピペで貼り付け

リダイレクトし忘れた場合や、別ウィンドウで実行した場合は、**計算完了後** にターミナル出力をすべて選択してコピーし、所定の `.log` に貼り付ける。

スクリプト	貼り付け先
feynman_eml_sr_fable_test2.py	results/feynman_fable_test2.log
feynman_eml_sr_fable_test2_v2.py	results/feynman_fable_test2_v2.log

手順:

- 実行が完了し、末尾に `SUMMARY:` と `[INFO] Results saved to ...` が表示されていることを確認する。
- ターミナルで **先頭（==== のヘッダー）から末尾（サマリー行）まで** をすべて選択してコピーする。
- エディタで上記 `.log` を開き、**既存の内容を上書き**（空ファイルならそのまま貼り付け）する。
- 保存する。

貼り付ける内容の例（`results/feynman_fable_test1.log` と同形式）：

```
=====
EML-SR Fable: Feynman Equations [2.test]
BEAM_WIDTH=2000 N_SAMPLES=1500 MAX_COMPLEXITY=8
=====
[INFO] Running 99 equations

[001/99] I.6.2a (vars=1)
Found: ...
RMSE: ... Status: ok Time: ...s
...
=====
SUMMARY: ... / 99 equations recovered
...
=====
```

`test2_v2` の場合はヘッダーが `[2.test_v2 noise=1%]` になる。貼り付け先は `results/feynman_fable_test2_v2.log` と混同しないこと。

7.3 JSON の内容（要約）

--

```
{
  "total": 99,
  "ok": ...,
  "partial": ...,
  "failed": ...,
  "total_elapsed_s": ...,
  "comparisons": { "first_AI": {...}, "cursor": {...} },
  "settings": { "beam_width": 2000, "N_SAMPLES": 1500, ... },
  "results": [
    {
      "eq_id": "I.12.1",
      "status": "ok",
      "found_formula": "...",
      "found_python": "...",
      "rmse": ...,
      "train_rmse": ...,
      "test_rmse": ...,
      "elapsed_s": ...,
      "candidates": [...]
    }
  ]
}
```

8. 結果の共有（git add / commit / push）

計算完了後、次のファイルをリポジトリに反映して push する。

8.1 コミット対象ファイル

種類	パス
JSON（test2）	results/eml_sr_fable_feynman_test2_results.json
JSON（test2_v2）	results/eml_sr_fable_feynman_test2_v2_results.json
レポート（test2）	texts/eml_sr_fable_feynman_test2_report.md
レポート（test2_v2）	texts/eml_sr_fable_feynman_test2_v2_report.md
実行ログ（test2）	results/feynman_fable_test2.log
実行ログ（test2_v2）	results/feynman_fable_test2_v2.log

スモークのみ実行した場合は *_smoke.json / *_smoke.md を代わりにコミットしてよい。

8.2 手順

リポジトリルートで:

```
git status

git add results/eml_sr_fable_feynman_test2_results.json
git add results/eml_sr_fable_feynman_test2_v2_results.json
git add texts/eml_sr_fable_feynman_test2_report.md
git add texts/eml_sr_fable_feynman_test2_v2_report.md
git add results/feynman_fable_test2.log
git add results/feynman_fable_test2_v2.log

git commit -m "Feynman 2.test / 2.test_v2 ベンチマーク結果を追加"

git push origin claudel/eml-sr-fable-algorithm-7w8j49
```

ブランチ名は作業中のブランチに合わせて変更すること。main へ直接 push しない（未マージの作業ブランチ上で実行する想定）。

8.3 別 PC のみで実行した場合

別 PC で clone → 実行 → 上記 commit / push まで行えば、元の PC では git pull で結果を取得できる。

9. トラブルシューティング

症状	対処
ModuleNotFoundError: eml_sr_fable	venv を activate したうえで <code>cd eml-sr_fable && maturin develop --release --features python,full-math</code> を再実行
ModuleNotFoundError: feynman_fable_common	src/ から実行しているか確認
No module named 'numpy' / pandas	<code>pip install numpy pandas</code>
Rust ビルド失敗 (Windows)	VS Build Tools をインストールし、ターミナルを再起動
FeynmanEquations.csv がない	ブランチが正しいか確認。 <code>git checkout claude/eml-sr-fable-algorithm-7w8j49</code>
1 式が 600 秒で打ち切られる	2.test の設計どおり (<code>time_budget_s=600</code>)。正常動作

10. チェックリスト (最短経路)

```
❑ git clone
❑ git checkout claude/eml-sr-fable-algorithm-7w8j49
❑ python -m venv .venv && activate
❑ pip install maturin numpy pandas
❑ cd eml-sr_fable && maturin develop --release --features python,full-math
❑ python -c "import eml_sr_fable"
❑ cd ../src && python feynman_eml_sr_fable_test2.py --smoke
❑ cd ../src && python feynman_eml_sr_fable_test2_v2.py --smoke
❑ 問題なければ本番: python feynman_eml_sr_fable_test2.py
❑ 続けて: python feynman_eml_sr_fable_test2_v2.py
❑ ターミナル出力を results/feynman_fable_test2.log にコピペ (またはリダイレクト済みなら確認)
❑ ターミナル出力を results/feynman_fable_test2_v2.log にコピペ (同上)
❑ git add (JSON / レポート / .log)
❑ git commit
❑ git push
```

作成日: 2026-07-07